

Module 5 - Spring Framework & Spring Boot

1. Introduction to Spring Framework

The **Spring Framework** is one of the most popular open-source frameworks used for developing Java applications.

Spring was created by **Rod Johnson** in 2003 to overcome the complexity of Enterprise JavaBeans (EJB).

The main objective of the Spring Framework is to make Java development simpler by promoting **loose coupling**, **dependency injection**, and **modular programming**.

Instead of developers manually creating objects and managing dependencies, Spring automatically creates and manages them using its **IoC (Inversion of Control) Container**. This reduces code complexity, improves maintainability, and makes applications easier to test.

Spring follows the principle of "**Write Less Code, Build Better Applications.**"

It provides various modules such as **Spring Core**, **Spring MVC**, **Spring Data**, **Spring Security**, **Spring Boot**, and **Spring AOP**, allowing developers to use only the components required for their application.

Spring offers the following benefits:

- Reduces boilerplate code.
- Promotes loose coupling.
- Simplifies testing using Dependency Injection.
- Provides built-in transaction management.
- Integrates easily with Hibernate, JPA, Maven, and Spring Boot.
- Supports web applications, REST APIs, and microservices.

Spring Modules

Module	Purpose
Spring Core	Provides IoC and Dependency Injection.
Spring AOP	Implements Aspect-Oriented Programming.
Spring MVC	Develops web applications using the MVC pattern.
Spring Data	Simplifies database operations.
Spring ORM	Integrates with Hibernate and JPA.
Spring Security	Provides authentication and authorization.
Spring Boot	Simplifies Spring application development.

Inversion of Control (IoC)

It means that the responsibility of creating and managing objects is transferred from the programmer to the Spring Framework.

In traditional Java programming, developers manually create objects using the `new` keyword. This results in tightly coupled code because classes directly depend on one another. In Spring, the IoC container creates and manages objects automatically. Developers simply request the required object, and Spring provides it.

By implementing IoC, applications become loosely coupled, making them easier to test, maintain, and extend.

Dependency Injection (DI)

It is a design pattern in which the objects required by a class are provided by an external source instead of being created by the class itself. In Spring, the **IoC Container** is responsible for creating objects and injecting their dependencies automatically.

Without Dependency Injection, classes become tightly coupled because they create their own dependent objects using the `new` keyword. This makes the application difficult to maintain and test. By using DI, classes become loosely coupled, allowing developers to replace or modify dependencies without changing the existing code.

Dependency Injection improves flexibility, reusability, and testability. Since objects are managed by the Spring container, developers can easily switch implementations or use mock objects during unit testing.

Types of Dependency Injection

Spring mainly supports two types of Dependency Injection:

- Constructor Injection
- Setter Injection

Constructor Injection

Constructor Injection is the process of injecting dependencies through a class constructor. The required **objects are passed as constructor arguments** when the object is created.

It is the **recommended approach** because it ensures that required dependencies are available when the object is created. It also makes objects immutable and easier to test.

Example

Engine.java

```
public class Engine {  
    public void start() {  
        System.out.println("Engine Started");  
    }  
}
```

Car.java

```
public class Car {  
    private Engine engine;  
    public Car(Engine engine) {  
        this.engine = engine;  
    }  
    public void drive() {  
        engine.start();  
        System.out.println("Car is Moving");  
    }  
}
```

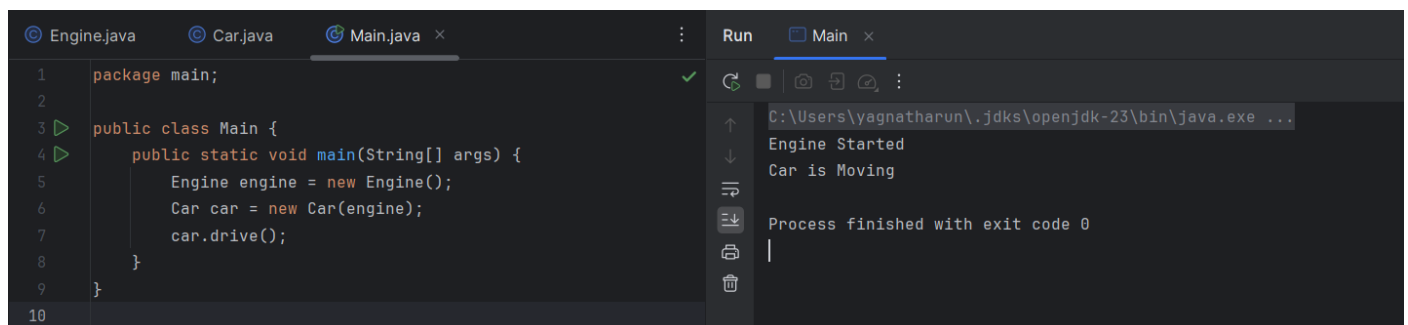
Main.java

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Engine engine = new Engine();  
  
        Car car = new Car(engine);  
  
        car.drive();  
  
    }  
  
}
```

Output

Engine Started

Car is Moving



```
1 package main;  
2  
3 public class Main {  
4     public static void main(String[] args) {  
5         Engine engine = new Engine();  
6         Car car = new Car(engine);  
7         car.drive();  
8     }  
9 }  
10
```

Run Main x

C:\Users\yagnatharun\jdk\openjdk-23\bin\java.exe ...
Engine Started
Car is Moving
Process finished with exit code 0

Setter Injection

Setter Injection is the process of injecting dependencies using setter methods after the object has been created.

It is useful when dependencies are optional or may change during runtime.

Engine.java

```
public class Engine {  
  
    public void start() {  
  
        System.out.println("Engine Started");  
  
    }  
  
}
```

Car.java

```
public class Car {  
  
    private Engine engine;  
  
    public void setEngine(Engine engine) {  
  
        this.engine = engine;  
  
    }  
  
    public void drive() {  
  
        engine.start();  
  
    }  
  
}
```

```

        System.out.println("Car is Moving");
    }
}

```

Main.java

```

public class Main {

    public static void main(String[] args) {

        Engine engine = new Engine();

        Car car = new Car();

        car.setEngine(engine);

        car.drive();

    }

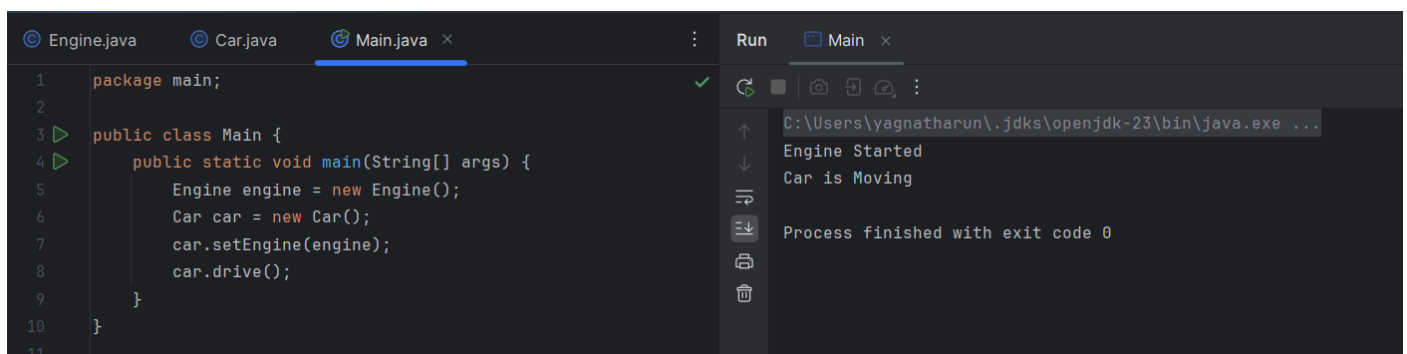
}

```

Output

Engine Started

Car is Moving



Constructor Injection vs Setter Injection

Constructor Injection	Setter Injection
Dependency is mandatory.	Dependency can be optional.
Object is immutable.	Object can be modified later.
Recommended in Spring.	Used for optional dependencies.
Dependency provided during object creation.	Dependency provided after object creation.

Spring IoC Container

It is responsible for creating, configuring, managing, and destroying Java objects known as **Beans**. Instead of developers manually creating objects, the IoC Container creates and manages them automatically based on the application configuration.

The IoC Container reads configuration information from XML files, Java configuration classes, or annotations. It then creates the required objects and injects dependencies wherever needed. This greatly simplifies application development by reducing object creation code.

The Spring Framework provides two main types of IoC containers:

- BeanFactory
- ApplicationContext

BeanFactory

BeanFactory is the basic implementation of the Spring IoC Container. It creates objects only when they are requested. This approach is called **lazy loading** because beans are initialized only when needed.

BeanFactory consumes less memory and is suitable for simple applications. However, it lacks many advanced features available in ApplicationContext.

Features of BeanFactory

- Supports lazy initialization.
- Lightweight.
- Suitable for small applications.
- Provides basic dependency injection.

ApplicationContext

ApplicationContext is an advanced implementation of the Spring IoC Container. It extends BeanFactory and provides many additional features such as event handling, internationalization, automatic bean initialization, and annotation support.

Unlike BeanFactory, ApplicationContext creates all singleton beans when the application starts. This is known as **eager initialization**.

Most Spring applications use **ApplicationContext** because it offers more functionality and better integration with Spring components.

Features of ApplicationContext

- Supports eager initialization.
- Event handling.
- Internationalization (i18n).
- Annotation support.
- Automatic Bean creation.

Spring Beans

A **Bean** is a Java object that is created, configured, and managed by the Spring IoC Container. Instead of creating objects manually using the new keyword, developers define beans in the Spring configuration, and the container manages their lifecycle automatically.

Beans can represent any object in an application, such as services, repositories, controllers, or utility classes. The Spring container is responsible for instantiating these beans, injecting their dependencies, and managing their lifecycle from creation to destruction.

Bean Lifecycle

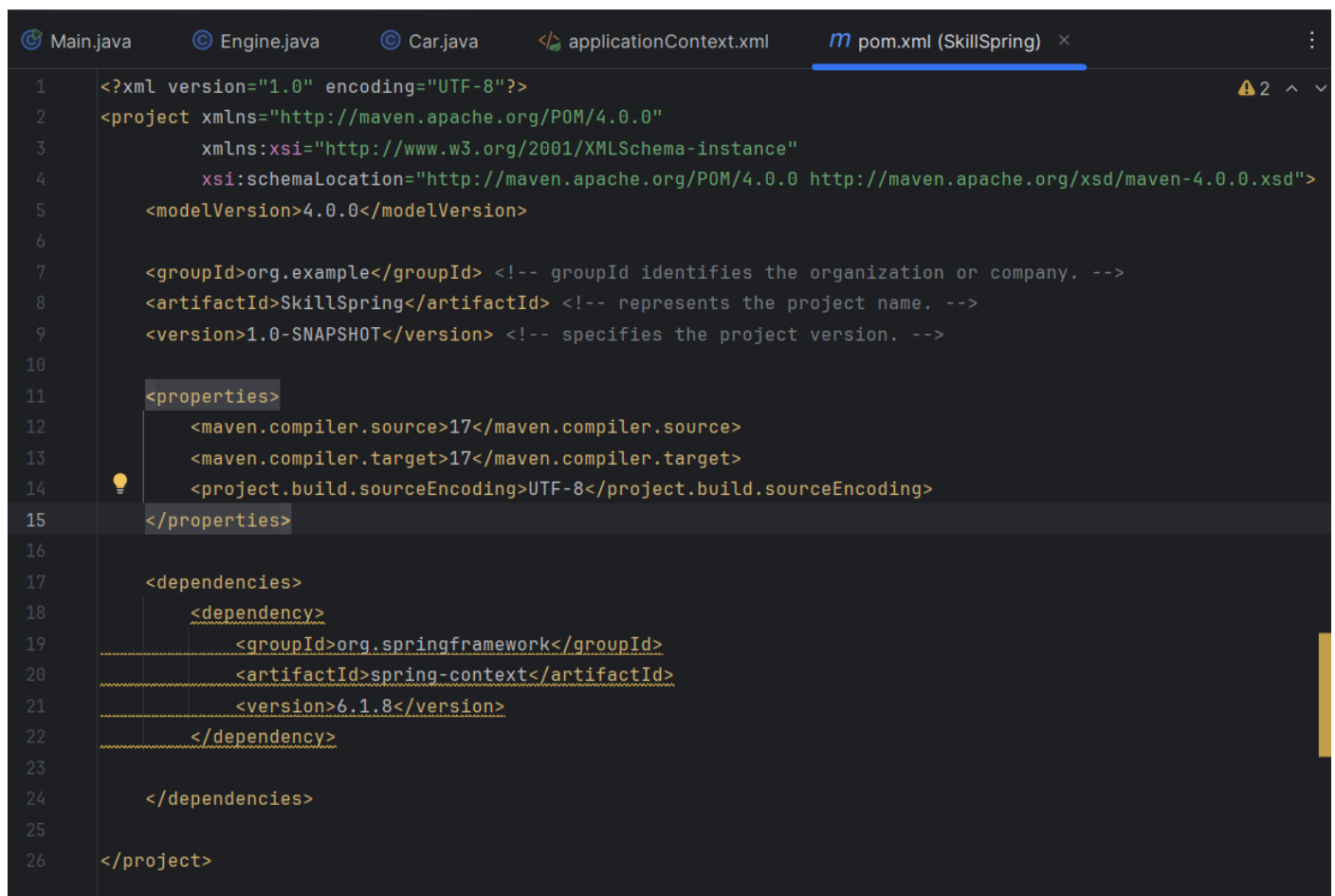
The lifecycle of a Spring Bean consists of several stages:

Bean Definition -> Object Creation -> Dependency Injection -> Initialization -> Bean Ready to Use -> Bean Destruction

Advantages of Spring Beans

- Managed automatically by Spring.
- Promotes loose coupling.
- Reduces object creation code.
- Supports dependency injection.
- Easy to configure and maintain.

Spring Bean Configuration

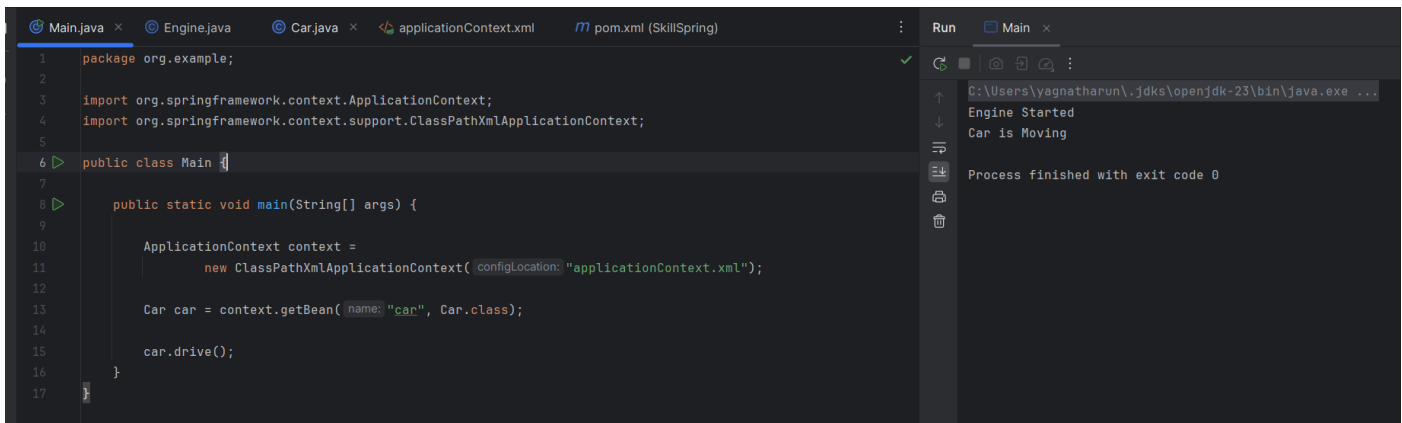


```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5      <modelVersion>4.0.0</modelVersion>
6
7      <groupId>org.example</groupId> <!-- groupId identifies the organization or company. -->
8      <artifactId>SkillSpring</artifactId> <!-- represents the project name. -->
9      <version>1.0-SNAPSHOT</version> <!-- specifies the project version. -->
10
11      <properties>
12          <maven.compiler.source>17</maven.compiler.source>
13          <maven.compiler.target>17</maven.compiler.target>
14          <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
15      </properties>
16
17      <dependencies>
18          <dependency>
19              <groupId>org.springframework</groupId>
20              <artifactId>spring-context</artifactId>
21              <version>6.1.8</version>
22          </dependency>
23      </dependencies>
24
25
26  </project>
```

```
Main.java Engine.java Car.java applicationContext.xml x pom.xml (SkillSpring)
1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <beans xmlns="http://www.springframework.org/schema/beans"
4       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5
6       xsi:schemaLocation="
7         http://www.springframework.org/schema/beans
8         http://www.springframework.org/schema/beans/spring-beans.xsd">
9
10      <bean id="engine"
11            class="org.example.Engine"/>
12
13      <bean id="car"
14            class="org.example.Car">
15
16          <property name="engine"
17                    ref="engine"/>
18
19      </bean>
20
21 </beans>
```

```
Main.java Engine.java Car.java x applicationContext.xml pom.xml (SkillSpring)
1 package org.example;
2
3 public class Car { no usages
4
5     private Engine engine; 2 usages
6
7     public void setEngine(Engine engine) { no usages
8         this.engine = engine;
9     }
10
11     public void drive() { no usages
12         engine.start();
13         System.out.println("Car is Moving");
14     }
15
16 }
17
```

```
Main.java Engine.java x Car.java applicationContext.xml pom.xml (SkillSpring)
1 package org.example;
2
3 public class Engine { no usages
4     public void start() { no usages
5         System.out.println("Engine Started");
6     }
7
8 }
```



Annotation-Based Bean Configuration

Spring provides annotations that eliminate the need for XML configuration. This approach is simpler and is widely used in modern Spring applications.

Common Spring Annotations

Annotation	Purpose
@Component	Creates a generic Spring Bean.
@Service	Represents the Service Layer.
@Repository	Represents the Repository Layer.
@Controller	Represents the Controller Layer.
@Configuration	Defines a Java configuration class.
@Bean	Creates and returns a Spring Bean.

Example using @Component

Student.java

```
import org.springframework.stereotype.Component;

@Component

public class Student {

    public void display() {

        System.out.println("Welcome to Spring Framework");

    }

}
```

AppConfig.java

```
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan("com.example")

public class AppConfig {

}
```


Main.java

```
import org.springframework.context.ApplicationContext;

import org.springframework.context.annotation.AnnotationConfigApplicationContext;


public class Main {

    public static void main(String[] args) {

        ApplicationContext context =

            new AnnotationConfigApplicationContext(AppConfig.class);

        Student student = context.getBean(Student.class);

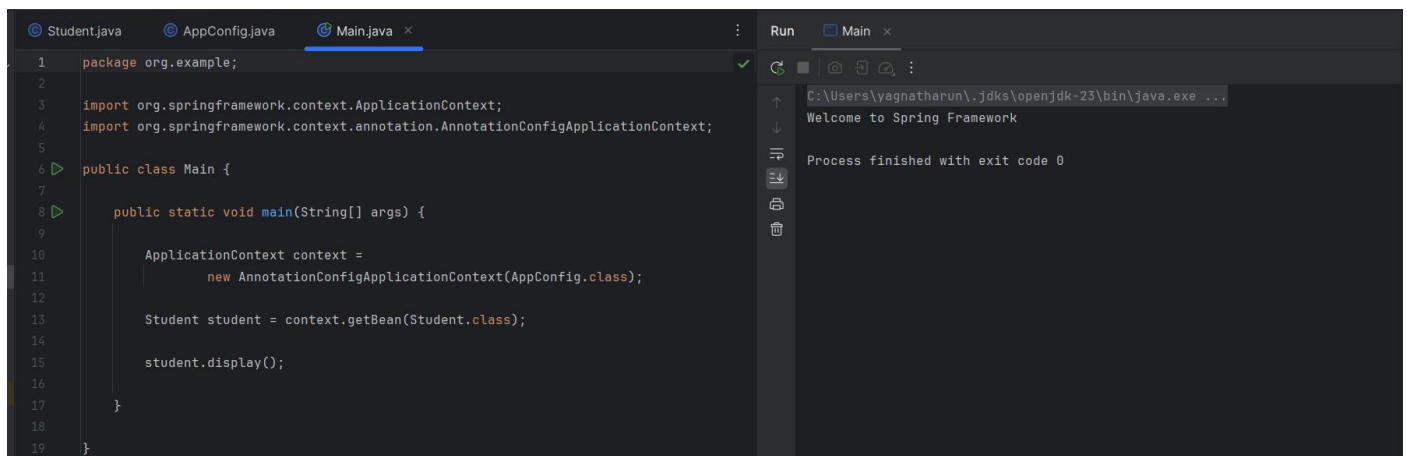
        student.display();

    }

}
```

Output

Welcome to Spring Framework



Autowiring in Spring

Autowiring is a feature of the Spring Framework that automatically injects the required dependency into a bean. Instead of manually creating objects or configuring them in XML, Spring identifies the required bean and injects it automatically using the **@Autowired** annotation.

Autowiring reduces the amount of configuration code and makes applications cleaner and easier to maintain. When the Spring IoC Container creates an object, it checks whether any dependencies are marked with @Autowired. If a suitable bean is found, Spring automatically injects it into the object.

Autowiring is commonly used in Spring Boot applications because it simplifies dependency management and promotes loose coupling between different layers of the application.

Engine.java

```
import org.springframework.stereotype.Component;

@Component

public class Engine {

    public void start() {
```

```
        System.out.println("Engine Started");
    }
}
```

Car.java

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component

public class Car {

    @Autowired

    private Engine engine;

    public void drive() {

        engine.start();

        System.out.println("Car is Moving");

    }

}
```

AppConfig.java

```
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan("org.example")
public class AppConfig {

}
```

Main.java

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {

    public static void main(String[] args) {

        ApplicationContext context =

            new AnnotationConfigApplicationContext(AppConfig.class);

        Car car = context.getBean(Car.class);

        car.drive();

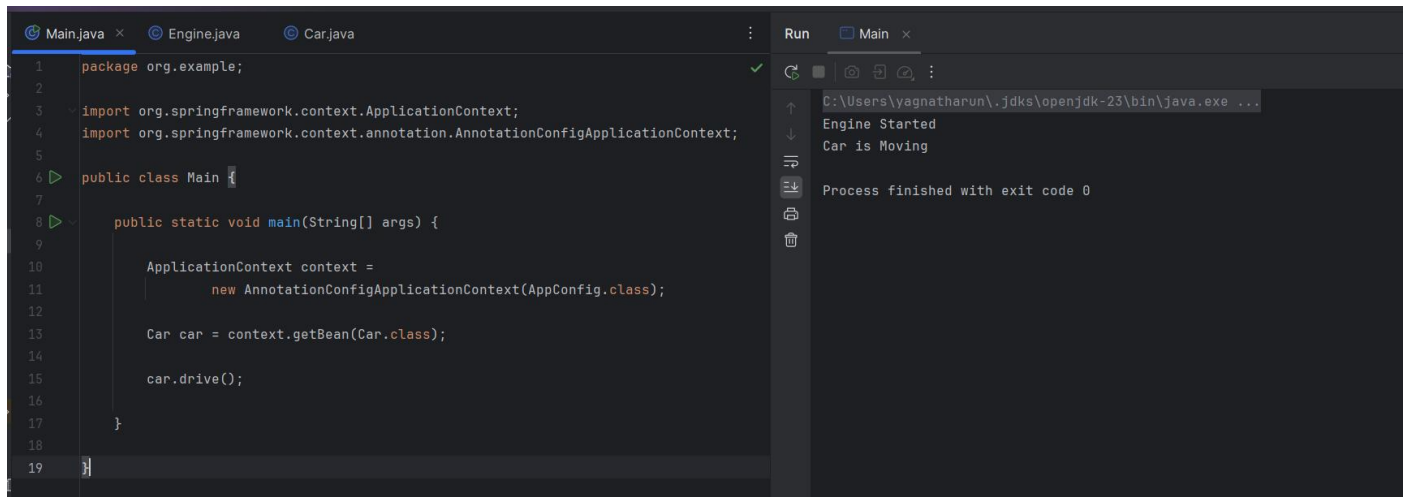
    }

}
```

Output

Engine Started

Car is Moving



The screenshot shows an IDE with three tabs: Main.java, Engine.java, and Car.java. The Main.java tab is active, displaying the following code:

```
1 package org.example;
2
3 import org.springframework.context.ApplicationContext;
4 import org.springframework.context.annotation.AnnotationConfigApplicationContext;
5
6 public class Main {
7
8     public static void main(String[] args) {
9
10         ApplicationContext context =
11             new AnnotationConfigApplicationContext(AppConfig.class);
12
13         Car car = context.getBean(Car.class);
14
15         car.drive();
16
17     }
18
19 }
```

The Run tab on the right shows the output of the application:

```
C:\Users\yagnatharun\.jdk\openjdk-23\bin\java.exe ...
Engine Started
Car is Moving
Process finished with exit code 0
```

@Qualifier Annotation

Sometimes multiple beans of the same type are available in the Spring container. In such situations, Spring cannot decide which bean should be injected and throws an exception called **NoUniqueBeanDefinitionException**.

The **@Qualifier** annotation is used to specify the exact bean that should be injected.

Engine.java

```
public interface Engine {

    void start();

}
```

PetrolEngine.java

```
import org.springframework.stereotype.Component;

@Component("petrol")

public class PetrolEngine implements Engine {

    public void start() {

        System.out.println("Petrol Engine Started");

    }

}
```

DieselEngine.java

```
import org.springframework.stereotype.Component;

@Component("diesel")

public class DieselEngine implements Engine {

    public void start() {

        System.out.println("Diesel Engine Started");

    }

}
```

Car.java

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Component;

@Component

public class Car {

    @Autowired
    @Qualifier("petrol")

    private Engine engine;

    public void drive() {

        engine.start();

        System.out.println("Car is Moving");

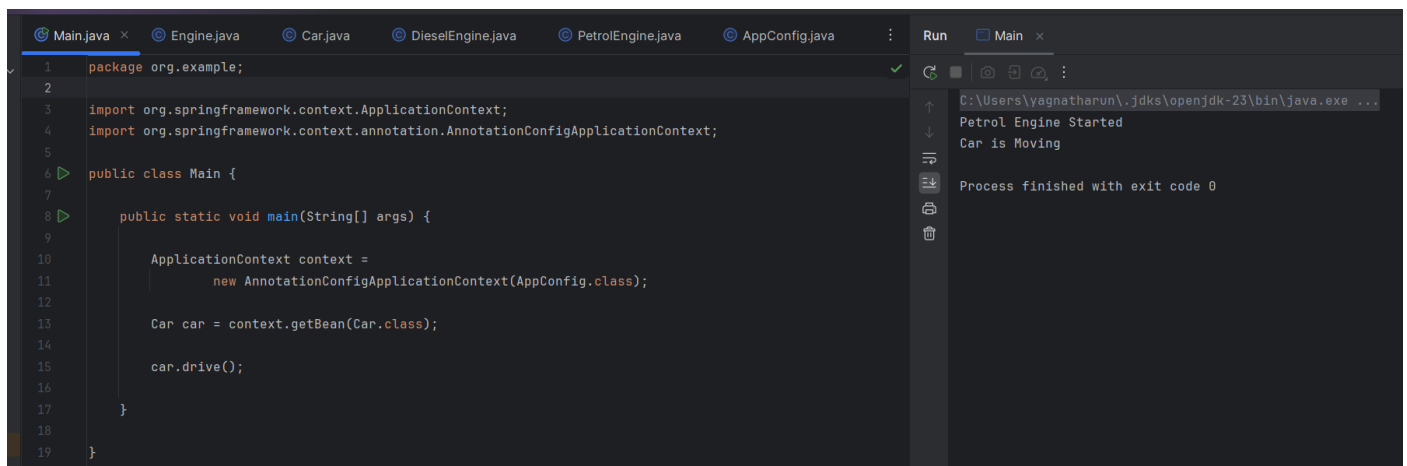
    }

}
```

Output

Petrol Engine Started

Car is Moving



```
1 package org.example;
2
3 import org.springframework.context.ApplicationContext;
4 import org.springframework.context.annotation.AnnotationConfigApplicationContext;
5
6 public class Main {
7
8     public static void main(String[] args) {
9
10         ApplicationContext context =
11             new AnnotationConfigApplicationContext(AppConfig.class);
12
13         Car car = context.getBean(Car.class);
14
15         car.drive();
16
17     }
18
19 }
```

Run Main ×

C:\Users\yagnatharun\.jdk\openjdk-23\bin\java.exe ...

Petrol Engine Started

Car is Moving

Process finished with exit code 0

@Primary Annotation

The **@Primary** annotation is used to specify the default bean that should be injected whenever multiple beans of the same type exist. Instead of using **@Qualifier** every time, developers can mark one bean as the default.

PetrolEngine.java

```
import org.springframework.context.annotation.Primary;
import org.springframework.stereotype.Component;

@Component

@Primary
```

```

public class PetrolEngine implements Engine {

    public void start() {

        System.out.println("Petrol Engine Started");

    }

}

```

DieselEngine.java

```

import org.springframework.stereotype.Component;

@Component

public class DieselEngine implements Engine {

    public void start() {

        System.out.println("Diesel Engine Started");

    }

}

```

Car.java

```

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.stereotype.Component;

@Component

public class Car {

    @Autowired

    private Engine engine;

    public void drive() {

        engine.start();

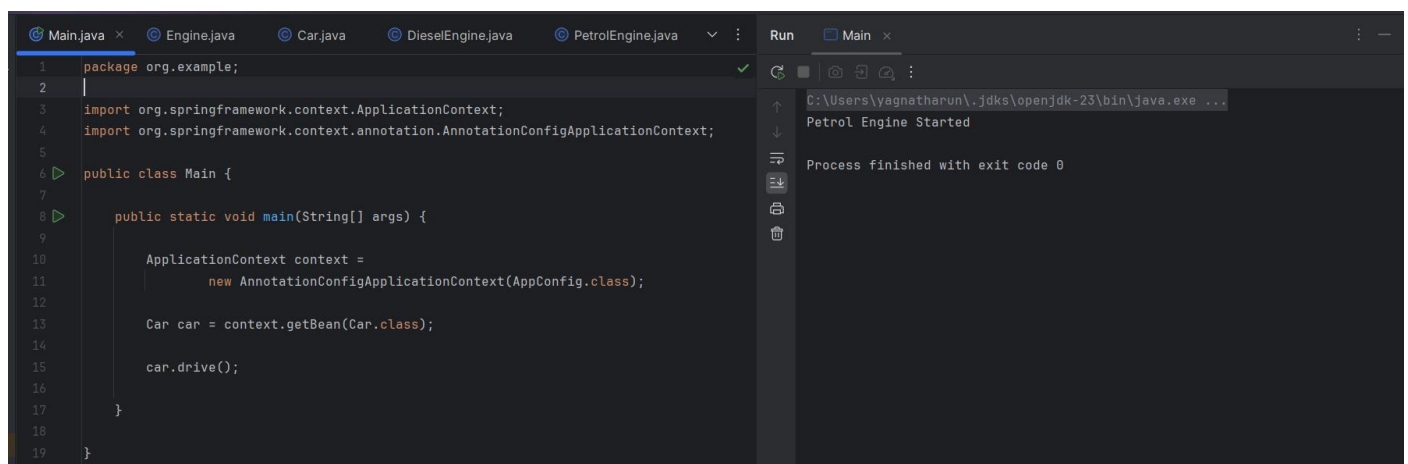
    }

}

```

Output

Petrol Engine Started



@Resource Annotation

The @Resource annotation is provided by Jakarta EE and is also supported by Spring. It performs dependency injection by matching the **bean name** rather than the bean type.

It is useful when developers want to inject a specific bean without using @Qualifier.

@Inject Annotation

The @Inject annotation is part of the Jakarta Dependency Injection specification. It performs the same function as @Autowired and is often used in applications that follow Java EE standards.

Unlike @Autowired, @Inject is not specific to Spring and can be used in any dependency injection framework that supports the Jakarta specification.

Spring AOP (Aspect-Oriented Programming)

Aspect-Oriented Programming (AOP) is a programming paradigm that helps developers separate cross-cutting concerns from the main business logic of an application. A cross-cutting concern is a functionality that is required by multiple modules, such as logging, security, transaction management, exception handling, and performance monitoring. Instead of writing the same code repeatedly in different classes, AOP allows developers to define it once and apply it wherever required.

In traditional programming, if every service method needs logging or security checks, developers must write the same code in every method. This results in code duplication and makes the application difficult to maintain. Spring AOP solves this problem by applying additional functionality automatically without modifying the existing business logic.

Spring AOP works by creating proxy objects around the target object. Whenever a method is called, the proxy intercepts the call, executes the additional functionality (called an **Aspect**), and then invokes the original method. This keeps the business logic clean and improves code maintainability.

Key Terminologies in AOP

Term	Description
Aspect	A class that contains cross-cutting logic.
Advice	The action performed by an aspect.
Join Point	A point during program execution where an aspect can be applied.
Pointcut	Expression that selects join points.
Target Object	The object being advised.
Proxy	Object created by Spring to apply aspects.

Types of Advice

Spring AOP provides different types of advice depending on when the additional code should execute.

Advice	Description
Before	Executes before the method.
After	Executes after the method.
After Returning	Executes after successful execution.
After Throwing	Executes when an exception occurs.
Around	Executes before and after the method.

EmployeeService.java

```
import org.springframework.stereotype.Service;
```

```
@Service
```

```
public class EmployeeService {  
    public void display() {  
        System.out.println("Employee Details");  
    }  
}
```

LoggingAspect.java

```
import org.aspectj.lang.annotation.Aspect;  
import org.aspectj.lang.annotation.Before;  
import org.springframework.stereotype.Component;
```

```
@Aspect
```

```
@Component
```

```
public class LoggingAspect {  
    @Before("execution(* EmployeeService.display(..))")  
    public void logBefore() {  
        System.out.println("Logging Started...");  
    }  
}
```

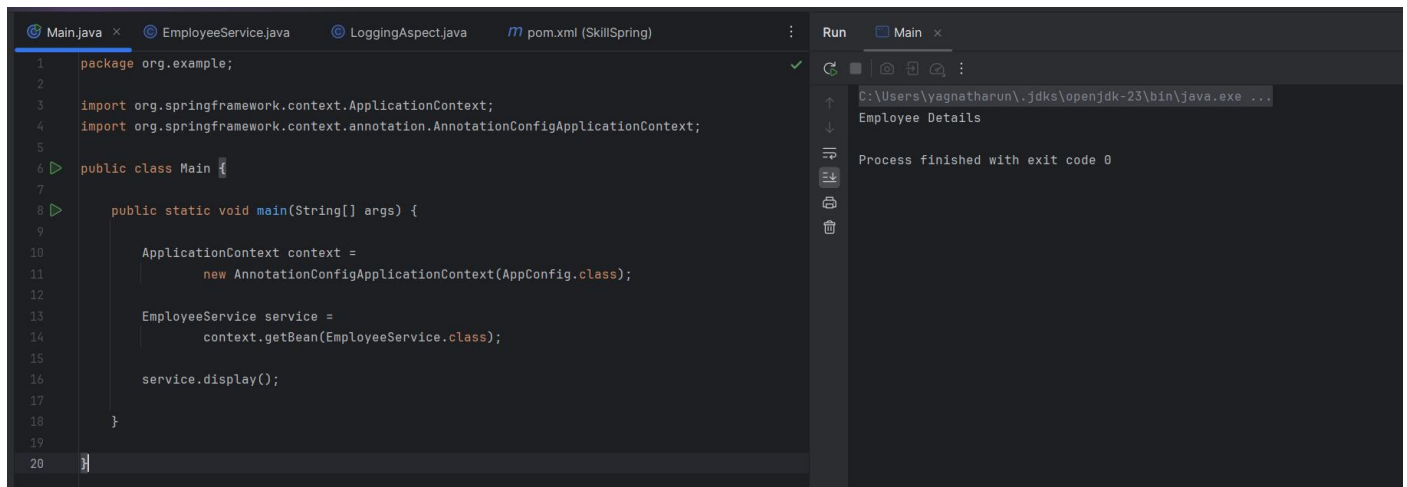
Main.java

```
import org.springframework.context.ApplicationContext;  
import org.springframework.context.annotation.AnnotationConfigApplicationContext;  
public class Main {  
    public static void main(String[] args) {  
        ApplicationContext context =  
            new AnnotationConfigApplicationContext(AppConfig.class);  
        EmployeeService service =  
            context.getBean(EmployeeService.class);  
        service.display();  
    }  
}
```

Output

Logging Started...

Employee Details



The screenshot shows an IDE with two panels. The left panel displays the source code for `Main.java`, which is part of the `org.example` package. It imports `org.springframework.context.ApplicationContext` and `org.springframework.context.annotation.AnnotationConfigApplicationContext`. The `main` method creates an `ApplicationContext`, gets an `EmployeeService` bean, and calls `display()`. The right panel shows the 'Run' output, indicating the process finished with exit code 0 and displaying the text 'Employee Details'.

```
1 package org.example;
2
3 import org.springframework.context.ApplicationContext;
4 import org.springframework.context.annotation.AnnotationConfigApplicationContext;
5
6 public class Main {
7
8     public static void main(String[] args) {
9
10         ApplicationContext context =
11             new AnnotationConfigApplicationContext(AppConfig.class);
12
13         EmployeeService service =
14             context.getBean(EmployeeService.class);
15
16         service.display();
17
18     }
19
20 }
```

Run Main x
C:\Users\yagnatharun\.jdk\openjdk-23\bin\java.exe ...
Employee Details
Process finished with exit code 0

Spring MVC (Model-View-Controller)

Spring MVC is a web framework provided by the Spring Framework for developing web applications following the **Model-View-Controller (MVC)** architectural pattern.

It separates an application into three independent components: **Model**, **View**, and **Controller**. This separation improves code organization, maintainability, and scalability.

When a user sends a request through a web browser, the request is first received by the **Controller**. The controller processes the request, communicates with the **Model** to perform business operations, and finally sends the processed data to the **View**, which generates the response displayed to the user.

Spring MVC simplifies web development by providing annotations such as `@Controller`, `@RequestMapping`, `@GetMapping`, and `@PostMapping`. These annotations reduce configuration and make request handling much easier.

MVC Architecture

User -> Controller -> Model -> Database -> View -> User

Components of MVC

Model

The Model represents the application's data and business logic. It interacts with the database and processes information requested by the controller.

View

The View is responsible for displaying data to the user. Common view technologies include JSP, Thymeleaf, and HTML.

Controller

The Controller receives client requests, invokes the required business logic, and returns the appropriate view.

Example

HomeController.java

```
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller

public class HomeController {

    @GetMapping("/home")

    public String home() {

        return "home";

    }

}
```

home.html

```
<!DOCTYPE html>

<html>

<head>

    <title>Spring MVC</title>

</head>

<body>

<h2>Welcome to Spring MVC</h2>

</body>

</html>
```

Output

http://localhost:8080/home

Welcome to Spring MVC

